

LoRa 바이트 직렬화 프로토콜 v2

정본: 26-04-11 session 기반 동작 총정리본 §1-§8 본 문서는 session JSON 필드 정의를 LoRa 무선 전송용 고정 16바이트 레이아웃으로 매핑한다. 바이트 순서: **Big-Endian** (네트워크 바이트 순서, Modbus PDU와 동일)

1. 프레임 개요

모든 메시지는 고정 16 bytes.

Offset	Size	필드	설명
[0-1]	2B	node_id	수신 대상 노드 (u16 BE) — 비암호화
— 이하 암호화 구간			
[2]	1B	packet_id	패킷 식별 ID (u8, 0-255)
[3]	1B	type_module	상위 4bit: type, 하위 4bit: module_type
[4-15]	12B	body	본문 (미사용 바이트는 0x00 패딩)

- 헤더: 4 bytes (node_id 2B + packet_id 1B + type_module 1B)
- 본문: 12 bytes
- 암호화 구간: 14 bytes ([2]-[15])
- packet_id는 0-255 순환. 재전송 시 동일 값 유지. ACK는 원본과 동일 packet_id.

type_module 바이트 인코딩

```
type_module = (type_code << 4) | module_type_code
```

예: DATA + power → (0x6 << 4) | 0x3 = 0x63

예: ACK + valve_0 → (0x0 << 4) | 0x0 = 0x00

2. 코드북 (Enum)

2.1 type_code (상위 4bit)

값	type	방향	ACK	재전송
0x0	ACK	both	X	X
0x1	CREATE	down	O	O
0x2	DELETE	down	O	O
0x3	NOTIFY	up	O	O
0x4	REQUEST	up	O	O

값	type	방향	ACK	재전송
0x5	UPDATE	down	O	O
0x6	DATA	up	O	O
0x7	END	up	O	O
0x8	PROGRESS	up	X	X
0x9-0xF	reserved	—	—	—

2.2 module_type_code (하위 4bit)

값	module_type	설명
0x0	valve_0	밸브 채널 0
0x1	valve_1	밸브 채널 1
0x2	sensor	RS485 슬롯 (device_index로 구분)
0x3	power	전원·배터리
0x4	system	비휘발 설정 세션
0x5	lora	LoRa 모듈 진단
0x6-0xF	reserved	향후 확장

2.3 notify_code (NOTIFY body[0])

NOTIFY 메시지는 type+module만으로 세부 종류를 구분할 수 없으므로 body[0]으로 구분한다.

값	이름	module_type	설명
0x01	valve_opened	valve_*	밸브 개방 완료
0x02	valve_closed	valve_*	밸브 폐쇄 완료
0x03	valve_failed	valve_*	밸브 제어 실패
0x04	battery_low	power	저전압 진입 (NORMAL→LOW)
0x05	battery_recover	power	정상 복구 (LOW→NORMAL)

2.4 reason_code (1 byte)

값	이름	용도
0x00	delete	서버 DELETE에 의한 정상 종료
0x01	timeout	타임아웃
0x02	failed	실패
0x03	aborted	취소/중단

값	이름	용도
0x04	low_battery	저전압
0x05	volume	유량 목표 도달
0x06	time	시간 목표 도달
0x07	both	유량+시간 동시
0x08	applied	설정 적용 성공
0x09	downlink_queue_full	하향 큐 포화
0x0A	timer_queue_full	타이머 큐 포화
0x0B	module_inactive	모듈 비활성
0x0C	overflow	오버플로

3. 메시지별 바이트 레이아웃

공통 헤더 [0-3]은 모든 메시지에서 동일. body [4-15] 부분만 기술한다.

3.1 ACK — 모든 모듈

```
type_module: 0x0? (ACK + 원본 module_type)
```

Offset	Size	필드	설명
4-15	12B	(padding)	전부 0x00

3.2 CREATE valve_*

```
type_module: 0x10 | valve_channel
```

Offset	Size	필드	범위	설명
4	1B	flags	bit0: request_on	1=목표도달 시 REQUEST 발송
5-6	2B	target_time	u16	목표 시간 (분), 0=미사용
7-8	2B	target_volume	u16	목표 유량 (L), 0=미사용
9-10	2B	timeout	u16	타임아웃 (분), 0=미사용
11-15	5B	(padding)	—	0x00

3.3 DELETE valve_*

```
type_module: 0x20 | valve_channel  
body: 없음 (전부 0x00)
```

3.4 NOTIFY valve_*

```
type_module: 0x30 | valve_channel
```

Offset	Size	필드	범위	설명
4	1B	notify_code	§2.3	0x01=opened, 0x02=closed, 0x03=failed
5	1B	detail	—	opened/closed: bit0=flow_confirmed. failed: reason_code
6–15	10B	(padding)	—	0x00

- valve_opened (0x01): detail bit0 = flow_open_confirmed
- valve_closed (0x02): detail bit0 = flow_closed_confirmed
- valve_failed (0x03): detail = reason_code (§2.4)

3.5 NOTIFY power

```
type_module: 0x33 (NOTIFY + power)
```

Offset	Size	필드	범위	설명
4	1B	notify_code	§2.3	0x04=battery_low, 0x05=battery_recover
5–15	11B	(padding)	—	0x00

3.6 REQUEST valve_close_request

```
type_module: 0x40 | valve_channel
```

Offset	Size	필드	범위	설명
4	1B	reason	reason_code	발생 사유 (time/volume/both)
5–6	2B	time_elapse	u16	경과 시간 (분)
7–8	2B	volume_elapse	u16	누적 유량 (L)

Offset	Size	필드	범위	설명
9-15	7B	(padding)	—	0x00

3.7 UPDATE valve_update

```
type_module: 0x50 | valve_channel
```

Offset	Size	필드	범위	설명
4	1B	flags	bit0: request_on	CREATE와 동일 필드
5-6	2B	target_time	u16	목표 시간 (분)
7-8	2B	target_volume	u16	목표 유량 (L)
9-10	2B	timeout	u16	타임아웃 (분)
11-15	5B	(padding)	—	0x00

3.8 PROGRESS valve_status

```
type_module: 0x80 | valve_channel
```

Offset	Size	필드	범위	설명
4-5	2B	time_elapse	u16	경과 시간 (분)
6-7	2B	volume_elapse	u16	누적 유량 (L)
8-15	8B	(padding)	—	0x00

3.9 END valve_end

```
type_module: 0x70 | valve_channel
```

Offset	Size	필드	범위	설명
4	1B	reason	reason_code	delete/timeout/failed
5-6	2B	time_elapse	u16	총 경과 시간 (분)
7-8	2B	volume_elapse	u16	총 누적 유량 (L)
9-15	7B	(padding)	—	0x00

3.10 CREATE sensor

```
type_module: 0x12 (CREATE + sensor)
```

Offset	Size	필드	범위	설명
4	1B	device_index	0-9	RS485 슬롯 번호
5-15	11B	(padding)	—	0x00

3.11 DELETE sensor

```
type_module: 0x22 (DELETE + sensor)
```

Offset	Size	필드	범위	설명
4	1B	device_index	0-9	RS485 슬롯 번호
5-15	11B	(padding)	—	0x00

3.12 END sensor_slot_end

```
type_module: 0x72 (END + sensor)
```

Offset	Size	필드	범위	설명
4	1B	device_index	0-9	RS485 슬롯 번호
5	1B	reason	reason_code	delete/failed
6-15	10B	(padding)	—	0x00

3.13 DATA rs485_collect

```
type_module: 0x62 (DATA + sensor)
```

RS485 읽기 결과를 보고한다. 최대 5개 레지스터, 각 레지스터 데이터는 최대 2바이트.

Offset	Size	필드	범위	설명
4	1B	device_index	0-9	RS485 슬롯 번호
5	1B	reg_mask	bitmask 5bit	bit0=reg0, ..., bit4=reg4. 1=성공/유효

Offset	Size	필드	범위	설명
6-7	2B	reg0	u16 BE	레지스터 0 값 (reg_mask bit0=0이면 0x0000)
8-9	2B	reg1	u16 BE	레지스터 1 값
10-11	2B	reg2	u16 BE	레지스터 2 값
12-13	2B	reg3	u16 BE	레지스터 3 값
14-15	2B	reg4	u16 BE	레지스터 4 값

- **reg_mask**: 비트 j=1이면 reg_j 데이터가 유효 (FC03 읽기 성공)
- 비트 j=0이면 해당 reg_j 슬롯은 0x0000 (실패 또는 미사용)
- 최대 5개 레지스터 × 2바이트 = 10바이트 → body 12B 안에 수용
- 향후 write register 결과도 동일 포맷 사용 가능 (reg_mask로 구분)

3.14 DATA battery_mv

type_module: 0x63 (DATA + power)

Offset	Size	필드	범위	설명
4-5	2B	battery_mv	u16 BE	전압 (mV), 0-65535
6-15	10B	(padding)	—	0x00

3.15 CREATE system

type_module: 0x14 (CREATE + system)

Offset	Size	필드	범위	설명
4	1B	target_module	module_type_code	설정 대상 모듈
5	1B	part_total	u8	UPDATE 조각 수
6-7	2B	cfg_base_addr	u16 BE	블록 시작 홀딩 주소
8-9	2B	cfg_word_count	u16 BE	연속 워드 수 (총 바이트=×2)
10-15	6B	(padding)	—	0x00

3.16 UPDATE config_transfer

type_module: 0x54 (UPDATE + system)

상세 포맷은 미정이나, **최대 10바이트** 이내로 확정 예정.

Offset	Size	필드	범위	설명
4	1B	part_index	u8	조각 인덱스 (0...part_total-1)
5	1B	data_len	u8	staging_data 실제 길이 (1-10)
6-15	10B	staging_data	u8[]	레지스터 이미지 조각 (data_len 이후 0x00)

조각당 최대 10바이트. body 12B 내 수용. 상세 바이트 해석은 target_module에 따라 정의 예정.

3.17 DELETE system

```
type_module: 0x24 (DELETE + system)
body: 없음 (커밋 트리거, 전부 0x00)
```

3.18 END system_config_end

```
type_module: 0x74 (END + system)
```

Offset	Size	필드	범위	설명
4	1B	reason	reason_code	applied/failed/aborted
5	1B	target_module	module_type_code	CREATE 때와 동일
6-9	4B	staging_crc32	u32 BE	CRC-32 (reason=applied 시 필수)
10-15	6B	(padding)	—	0x00

3.19 PROGRESS module_cmd_enqueue_failed

```
type_module: 0x85 (PROGRESS + lora)
```

Offset	Size	필드	범위	설명
4	1B	target_module	module_type_code	적재 실패한 대상 모듈
5	1B	reason_code	reason_code §2.4	실패 원인
6	1B	source_packet_id	u8	원 하향 packet_id (주기=0)
7-15	9B	(padding)	—	0x00

4. 메시지 요약표

#	type + module	type_module	body 사용 량	핵심 body 필드
1	ACK (모든 모듈)	0x0?	0B	—
2	CREATE valve	0x1?	7B	flags, target_time, target_volume, timeout
3	DELETE valve	0x2?	0B	—
4	NOTIFY valve	0x3?	2B	notify_code, detail
5	NOTIFY power	0x33	1B	notify_code
6	REQUEST valve	0x4?	5B	reason, time_elapse, volume_elapse
7	UPDATE valve	0x5?	7B	flags, target_time, target_volume, timeout
8	PROGRESS valve	0x8?	4B	time_elapse, volume_elapse
9	END valve	0x7?	5B	reason, time_elapse, volume_elapse
10	CREATE sensor	0x12	1B	device_index
11	DELETE sensor	0x22	1B	device_index
12	END sensor	0x72	2B	device_index, reason
13	DATA rs485	0x62	12B	device_index, reg_mask, reg0–4
14	DATA battery	0x63	2B	battery_mv
15	CREATE system	0x14	6B	target_module, part_total, cfg_base_addr, cfg_word_count
16	UPDATE config	0x54	2–12B	part_index, data_len, staging_data
17	DELETE system	0x24	0B	— (커밋 트리거)
18	END system	0x74	6B	reason, target_module, staging_crc32
19	PROGRESS lora	0x85	3B	target_module, reason_code, source_packet_id

? = valve 채널 번호 (0x0=valve_0, 0x1=valve_1) 전체 19종 메시지, 모두 16 bytes 고정

5. Hex 예시 (Round-trip 검증용)

5.1 ACK (valve_0, packet_id=7)

```

node_id pkt TM  body (all 0x00)
Hex: 00 1F 07 00 00 00 00 00 00 00 00 00 00 00 00

```

	node_id	pkt	TM	fl	t_time	t_vol	tout	padding
Hex:	00 1F	07	10	01	00 1E	03 E8	00 3C	00 00 00 00 00

```

node_id pkt TM nc dt padding
Hex: 00 1F 07 30 01 01 00 00 00 00 00 00 00 00 00
      |      └─ detail: bit0=1 (flow_open_confirmed)
      └─ notify_code: 0x01 (valve_opened)

```

[illegible]

```

node_id pkt TM bat_mv padding
Hex: 00 1F 66 63 33 90 00 00 00 00 00 00 00 00 00 00

```

	node_id	pkt	TM	di	mask	reg0	reg1	reg2	reg3	reg4
Hex:	00 1F	41	62	00	13	01 00	02 00	00 00	00 00	05 00
										└ reg4 유효
							└ reg1 유효			

```

└── reg0 유효
    └── 0x13 = 0b10011 (bit0,1,4 = 유효)

```

5.7 CREATE system (target=valve_0, parts=1, base=4096, words=8)

	node_id	pkt	TM	tg	pt	base	words	padding
Hex:	00 1F	C8	14	00	01	10 00	00 08	00 00 00 00 00 00

5.8 END system_config_end (reason=applied, target=valve_0, crc=0xDEADBEEF)

	node_id	pkt	TM	rs	tg	crc32	padding
Hex:	00 1F	CC	74	08	00	DE AD BE EF	00 00 00 00 00 00

5.9 PROGRESS module_cmd_enqueue_failed

	node_id	pkt	TM	tg	rc	src	padding
Hex:	00 1F	F5	85	00	09	58	00 00 00 00 00 00 00 00

└── source_packet_id=0x58
 └── reason=downlink_queue_full(0x09)
 └── target=valve_0(0x00)

6. 인코드/디코드

6.1 인코딩 (C)

```

void encode_header(uint8_t *buf, uint16_t node_id, uint8_t pkt_id,
                  uint8_t type_code, uint8_t module_code) {
    buf[0] = (node_id >> 8) & 0xFF;
    buf[1] = node_id & 0xFF;
    buf[2] = pkt_id;
    buf[3] = (type_code << 4) | (module_code & 0x0F);
    memset(&buf[4], 0, 12); // body 초기화
}

```

6.2 디코딩 (C)

```

uint16_t node_id    = (buf[0] << 8) | buf[1];
uint8_t packet_id   = buf[2];
uint8_t type_code   = (buf[3] >> 4) & 0x0F;
uint8_t module_code = buf[3] & 0x0F;

```

```
// body = &buf[4], 12 bytes
// type_code + module_code 조합으로 body 레이아웃 결정
```

6.3 검증 규칙

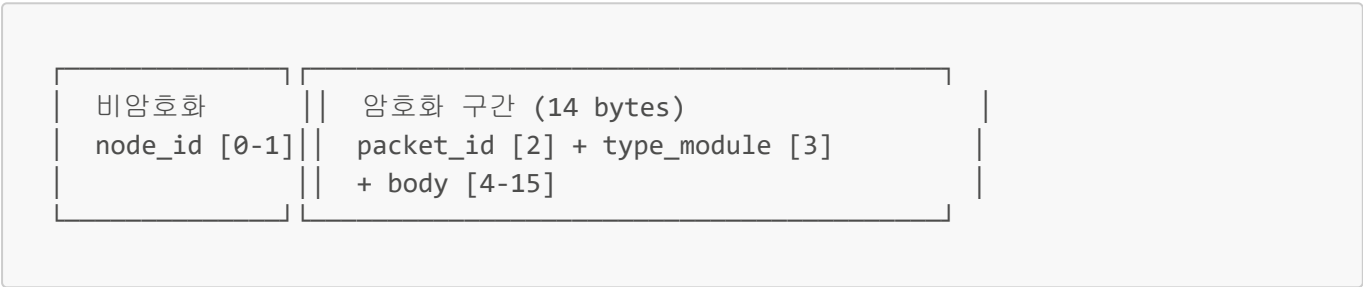
1. **프레임 길이**: 반드시 16 bytes
2. **type_code**: 0x0–0x8 범위, 그 외 → 에러
3. **module_type_code**: 0x0–0x5 범위, 그 외 → 에러
4. **ACK body 검사**: body[4–15] 전체 0x00
5. **NOTIFY notify_code**: body[4]가 type+module 허용 값인지 확인
6. **RS485 reg_mask**: 상위 3비트(bit5–7)는 0이어야 함
7. **round-trip**: encode(msg) → bytes → decode(bytes) → msg2, msg == msg2

7. session JSON ↔ 바이트 매핑표

session 필드	위치	크기	인코딩
node_id	[0–1]	u16 BE	그대로
packet_id	[2]	u8	0–255 순환
type	[3] 상위 4bit	4bit	§2.1
module_type	[3] 하위 4bit	4bit	§2.2
notify_type	body[0]	u8	§2.3 (NOTIFY만)
request_on	body bit0	1bit	bool→0/1
target_time	body u16	2B BE	분 단위
target_volume	body u16	2B BE	L 단위
timeout	body u16	2B BE	분 단위
battery_mv	body u16	2B BE	mV 단위
device_index	body u8	1B	0–9
reg_mask (step_ok)	body u8	1B	5bit 비트마스크
reg0–reg4	body u16×5	10B BE	레지스터 값 (고정 슬롯)
reason / reason_code	body u8	1B	§2.4
target_module	body u8	1B	§2.2 재사용
part_total	body u8	1B	1–255
part_index	body u8	1B	0–254
cfg_base_addr	body u16	2B BE	Modbus 홀딩 주소
cfg_word_count	body u16	2B BE	워드 수

session 필드	위치	크기	인코딩
staging_crc32	body u32	4B BE	IEEE 802.3 CRC-32
staging_data	body u8[]	≤10B	레지스터 이미지 조각
source_packet_id	body u8	1B	원 하향 packet_id
time_elapse	body u16	2B BE	분 단위
volume_elapse	body u16	2B BE	L 단위
flow_confirmed	body bit0	1bit	bool→0/1

8. 암호화 경계



9. v1 대비 변경 요약

항목	v1	v2
packet_id	2B (u16)	1B (u8)
sub_type 필드	헤더 1B 전용	삭제 (NOTIFY는 body[0]으로 구분)
헤더 크기	6B	4B
body 크기	10B (표준) / 25B (확장)	12B (고정)
프레임 종류	16B + 32B 2종	16B 단일
RS485 DATA	가변 길이 reg_bytes	고정 5슬롯 × 2B = 10B
config transfer	가변 staging_bytes (최대 24B)	최대 10B (body 내 수용)
총 프레임 크기	16B 또는 32B	16B 고정 통일